

Assignment 5: Advanced Multi-Client Chat Server using TCP

Objective

Enhance the multi-client TCP chat application developed in Assignment 4 by implementing advanced messaging features, improved client management, persistent chat history, and efficient server-side message routing. The assignment emphasizes concurrent programming, client-server architecture, state management, performance evaluation, and software evolution by extending an existing codebase.

Prerequisite

Students must reuse and extend their Assignment 4 implementation. Starting the application from scratch is not permitted.

Network Setup

Use Mininet with one server and four clients.

```
sudo mn --topo single,5
```

Topology:

h1 = Chat Server

h2 = Client A

h3 = Client B

h4 = Client C

h5 = Client D

Verify connectivity using:

nodes

net

pingall

Tasks (100 Marks)

Task 1: Enhanced Client Management (20 Marks)

Maintain username, IP address, port number, login time, and online/offline status for every connected client. Broadcast join and leave notifications. Display server statistics including connected users, messages processed, broadcast messages, and private messages.

Task 2: Private Messaging (25 Marks)

Implement private messaging using the format `/msg <username> <message>`. Only the intended recipient should receive the message. Display an appropriate error if the user does not exist.

Task 3: Online User List (10 Marks)

Implement a '/list' command that displays all currently connected users and updates dynamically.

Task 4: Persistent Chat History (10 Marks)

Store all messages in chat_history.csv with fields: timestamp, sender, receiver, message_type, message. When a user reconnects using the same username, display the last five messages sent by that user.

Task 5: Performance Evaluation (10 Marks)

Perform experiments with 2, 3, and 4 clients. Each client should send 50 messages. Measure average message delivery time, throughput, number of broadcast messages, and number of private messages. Store results in performance_results.csv.

Task 6: Wireshark Verification (10 Marks)

Capture TCP traffic using the filter 'tcp.port == 5000'. Include screenshots of client connection, broadcast message, private message, client disconnect, and TCP connection termination.

Task 7: Graph Generation (10 Marks)

Generate graphs for: (1) Number of Clients vs Average Delivery Time, (2) Number of Clients vs Throughput, and (3) Broadcast Messages vs Private Messages.

Task 8: Reflection Questions (5 Marks)

Answer briefly: (1) Why is server-side routing necessary for private messaging? (2) How does maintaining user state improve the application? (3) How would the application scale to 100 users? (4) Which Wireshark packets verified private messaging? (5) What improvements would you implement next?

Required Outputs

server.py
client.py
chat_history.csv
performance_results.csv
graphs/
screenshots/
report.pdf

CSV Formats

performance_results.csv:
clients,broadcast_messages,private_messages,avg_delay_ms,throughput_msgs_per_sec

chat_history.csv:
timestamp,sender,receiver,message_type,message

Graph Requirements

Generate:

- clients_vs_delay.png
- clients_vs_throughput.png
- message_type_distribution.png

Report Format

1. Objective
2. System Architecture
3. Design Improvements over Assignment 4
4. Program Design
5. Experimental Setup
6. Results
7. Graph Analysis
8. Wireshark Verification
9. Reflection Answers
10. Conclusion

Submission Guidelines

Submit:

ROLLNO_NAME_ASSIGNMENT5.zip

Folder Structure:

ROLLNO_NAME_ASSIGNMENT5/

```
├── server.py
├── client.py
├── chat_history.csv
├── performance_results.csv
├── graphs/
├── screenshots/
└── report.pdf
```

Evaluation Rubric

Component	Marks
Enhanced client management	20
Private messaging	25
Online user list	10
Persistent chat history	10
Performance evaluation	10
Wireshark verification	10
Graph generation	10
Reflection and report quality	5
Total	100

Important Notes

The application must run inside Mininet. Students must extend their Assignment 4 implementation. All screenshots, CSV files, graphs, and Wireshark captures must be generated from actual execution. Students should be prepared to demonstrate the application during evaluation.